written as an extension to the HTML markup you are already writing and is well suited to small interactions that don't need to manage a lot of state, or where the majority of the state is managed by the server application. A third tool, Strada, which manages interactions with native mobile devices, has not yet been released as I write this.

Many frameworks support the JSON interaction pattern and create their own markup on the client. We'll use React to represent those tools.[21] React replaces your HTML markup with a custom hybrid of JavaScript and HTML called JSX. React automatically updates the DOM when data changes, and is suited for interactions where a lot of state is stored on the client.

The code we'll write will be in TypeScript, which is a language that extends JavaScript by allowing us to specify the type of any variable, class attribute, function argument, or function return value.[22] I like using TypeScript because there are a whole bunch of mistakes I normally make in JavaScript that TypeScript catches for me. TypeScript's syntax is JavaScript with a few extra features. The first few chapters cover the TypeScript syntax as we need it, and then we'll take a look at TypeScript itself in more detail in Chapter 6, *TypeScript* and Chapter 13, *Validating Code with Advanced TypeScript*.

Finally, we'll use Rails bundling tools to put all these pieces together. These tools, jsbundling-rails, cssbundling-rails, and Propshaft, provide sensible defaults and useful conventions for where to put your JavaScript and other assets, allowing us to write it in a structure of our choosing and then package everything in a way that is easy for the browser to manage.

We'll start by creating a Rails 7.0 application with these tools, then show how much Hotwire and Turbo allow you to do without custom JavaScript. But first, before we dive in and write code, let's talk about web applications, Rails, and JavaScript for a second.